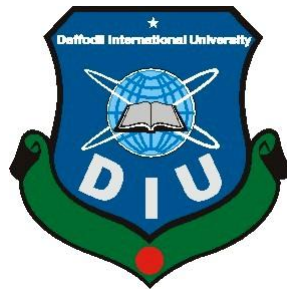


Aura Brew: The Ultimate Coffee Experience

MINI LAB PROJECT REPORT

This Report Presented in Partial Fulfillment of the course **CSE414: Mobile Application Design Lab** in the **Computer Science and Engineering Department**



DAFFODIL INTERNATIONAL UNIVERSITY

Dhaka, Bangladesh

April 18, 2026

DECLARATION

We hereby declare that this lab project has been done by us under the supervision of **Tanjir Ahmed Anik, lecturer**, Department of Computer Science and Engineering, Daffodil International University. We also declare that neither this project nor any part of this project has been submitted elsewhere as lab projects.

Submitted To:

Tanjir Ahmed Anik

Designation: Lecturer

Department of Computer Science and Engineering,
Daffodil International University

Submitted by

Most. Sumaiya Akter Sithi

ID:0242220005101915
Dept. of CSE, DIU

Sohi Bilkis Binte Yer

ID:0242220005101917
Dept. of CSE, DIU

Md Abir Ahmed Porosh

ID:0242220005101814
Dept. of CSE, DIU

COURSE & PROGRAM OUTCOME

The following course have course outcomes as following:.

Table 1: Course Outcome Statements

CO's	Statements
CO1	Have an understanding of different programming techniques, protocols available for development.
CO2	Have the abilities to analyse and describe the system requirements
CO3	Taking critical decision regarding client and server-side development
CO4	Be able to evaluate existing systems and build new systems by applying state of the art technology.

Table 2: Mapping of CO, PO, Blooms, KP and CEP

CO	PO	Blooms	KP	CEP
CO1	PO1	C1, C2	KP3	EP1, EP3
CO2	PO2	C2	KP3	EP1, EP3
CO3	PO3	C4, A1	KP3	EP1, EP2
CO4	PO3	C3, C6, A3, P3	KP4	EP1, EP3

The mapping justification of this table is provided in section 4.3.1, 4.3.2 and 4.3.3.

Table of Contents

Declaration	i
Course & Program Outcome	ii
1 Introduction	1
1.1 Introduction	1
1.2 Motivation	1
1.3 Objectives	1
1.4 Feasibility Study.....	2
1.5 Gap Analysis	2
1.6 Project Outcome.....	2
2 Proposed Methodology/Architecture	3
2.1 Requirement Analysis & Design Specification.....	3
2.1.1 Overview.....	3
2.1.2 Proposed Methodology/ System Design	3-
2.1.3 UI Design.....	4
2.2 Overall Project Plan.....	8
3 Implementation and Results	9
3.1 Implementation.....	9
3.2 Performance Analysis	9
3.3 Results and Discussion	9
4 Engineering Standards and Mapping	11
4.1 Impact on Society, Environment and Sustainability.....	11
4.1.1 Impact on Life.....	11
4.1.2 Impact on Society & Environment.....	11
4.1.3 Ethical Aspects	11
4.1.4 Sustainability Plan.....	11
4.2 Project Management and Team Work.....	11
4.3 Complex Engineering Problem.....	11
4.3.1 Mapping of Program Outcome.....	12
4.3.2 Complex Problem Solving	12
4.3.3 Engineering Activities.....	13
5 Conclusion	14
5.1 Summary	14
5.2 Limitation	14
5.3 Future Work.....	14
References	15

Chapter 1

Introduction

This chapter provides an overview of the background, purpose, workflow, and expected outcomes of the Aura Brew project a full-stack mobile application for premium coffee ordering and customization.

1.1 Introduction

Aura Brew is a comprehensive, full-stack mobile application designed to revolutionize the coffee ordering experience. Built using Flutter for the cross-platform mobile frontend and Django REST Framework for the backend API, the application provides users with a seamless, premium interface for browsing an extensive coffee and tea menu, customizing their beverages with specific milk options, toppings, and sizes, managing a smart cart, placing orders, and tracking their order history. The system incorporates a secure phone-based authentication mechanism using a 4-digit PIN, a membership reward system with bean tracking, and a sophisticated feedback collection module.

The project follows industry-standard software engineering practices, with a clean separation of concerns between the frontend and backend. The Flutter mobile client communicates with the Django REST API over HTTP using token-based authentication, ensuring data integrity and user security. The backend is cloud-deployed on Render with a serverless PostgreSQL database hosted on Neon.tech, and media assets are stored on Cloudinary, making the system production-ready and scalable.

The project repository is publicly available on GitHub at: <https://github.com/aaporosh/Aura-Brew>

1.2 Motivation

The global coffee industry has seen an unprecedented surge in the demand for digital ordering platforms. With the rise of mobile-first experiences, consumers increasingly prefer the convenience of browsing menus, customizing drinks, and placing orders from their smartphones rather than waiting in long queues at physical stores. Popular platforms like Starbucks and Dutch Bros have set a high standard for mobile coffee ordering, yet many local and mid-scale coffee brands still lack such digital infrastructure.

The motivation behind Aura Brew stems from the desire to create a production-grade mobile application that demonstrates the complete lifecycle of a modern full-stack mobile project — from UI/UX design to backend development, cloud deployment, and real-world API integration. This project serves as a comprehensive learning exercise and portfolio piece that showcases the developer's ability to build, deploy, and manage a complex software system using industry-standard technologies such as Flutter, Django, PostgreSQL, and cloud services.

Additionally, the project addresses the academic requirement of the Mobile Application Development (MAD) course by demonstrating practical implementation of mobile development concepts including state management, navigation, RESTful API consumption, secure authentication, responsive UI design, and cross-platform compatibility.

1.3 Objectives

The primary objectives of the Aura Brew project are:

- To design and develop a premium, user-friendly mobile application for coffee ordering using Flutter and Dart.
- To implement a robust RESTful API backend using Django REST Framework with PostgreSQL database support.
- To integrate secure phone-based authentication with a 4-digit PIN mechanism, replacing traditional passwords.
- To implement product browsing with category-based filtering, featured products, and detailed customization options (milk, toppings, sizes, sugar level).
- To develop a full shopping cart system with add, update, delete, and clear functionality.
- To create an order management system that converts cart items into orders with automatic price calculations (subtotal, tax, service fee).
- To implement a rewards/membership tier system (Bronze → Silver → Gold) based on cumulative purchases.
- To build user profile management with avatar upload, address management, and preference settings.
- To deploy the backend on Render (PaaS) with Neon.tech PostgreSQL and Cloudinary media storage for production readiness.
- To utilize Provider for efficient state management across the Flutter application.

1.4 Feasibility Study

The feasibility of the Aura Brew project was evaluated across four key dimensions:

Technical Feasibility:

The project utilizes well-established, open-source technologies. Flutter (Dart SDK ^3.9.2) provides a mature cross-platform mobile development framework with excellent documentation and community support. Django REST Framework (Django >=5.1, DRF >=3.15) is a battle-tested backend framework for building RESTful APIs. PostgreSQL is the world's most advanced open-source relational database. All tools and libraries used are actively maintained and have extensive community resources. The development team possesses the necessary skills in Flutter/Dart, Python/Django, and database management to execute the project successfully.

Operational Feasibility:

The application is designed for Android devices with plans for iOS support. The intuitive UI with onboarding screens, simple 4-digit PIN authentication, and familiar shopping patterns (browse →

customize → cart → checkout) ensures ease of use. The app requires minimal user training and follows established mobile UX conventions.

Economic Feasibility:

The entire technology stack is open-source and free to use. Render's free tier provides adequate hosting for development and demonstration purposes. Neon.tech offers a generous free tier for serverless PostgreSQL. Cloudinary's free plan supports sufficient media storage. The only cost is developer time, making this project economically viable for an academic setting.

Schedule Feasibility:

The project was planned to be completed within the academic semester timeframe. The modular architecture (separate frontend and backend) allowed parallel development and iterative refinement. Key milestones included UI design, backend API development, frontend-backend integration, and cloud deployment.

1.5 Gap Analysis

An analysis of existing coffee ordering applications reveals several gaps that Aura Brew aims to address:

Feature	Existing Apps	Aura Brew
Full-Stack Open Source	Most are proprietary, closed-source	Fully open-source on GitHub with clean architecture
PIN-Based Auth	Password/email based, complex	Simple 4-digit PIN with phone number — fast and secure
Drink Customization	Limited or basic options	Granular: milk type, toppings, size, sugar level slider
Rewards System	Often tied to paid memberships	Free tier-based (Bronze/Silver/Gold) with bean accumulation
Cloud Deployment	Tied to proprietary infrastructure	Render + Neon.tech + Cloudinary — modern serverless stack
Cross-Platform	Often platform-specific (iOS/Android)	Flutter-based: single codebase for Android, iOS, Web, Desktop
Admin Dashboard	Separate tools needed	Built-in Django Jazzmin admin with custom branding

1.6 Project Outcome

Upon successful completion, the Aura Brew project delivers the following outcomes:

- A fully functional, production-ready mobile application with 15+ screens covering the complete coffee ordering workflow.
- A RESTful API backend with 7 Django apps (accounts, products, orders, cart, feedback, stores, banners) and 20+ API endpoints.
- Secure token-based authentication with Django REST Framework's TokenAuthentication.
- Cloud-deployed backend accessible at <https://aura-brew.onrender.com> with PostgreSQL and Cloudinary integration.
- A downloadable Android APK available via GitHub Releases for immediate user testing.
- Demonstration of full-stack software engineering skills applicable to industry-level projects.

Chapter 2

Proposed Methodology/Architecture

This chapter explains the requirement analysis, design approach, system architecture, and UI planning for the Aura Brew project.

2.1 Requirement Analysis & Design Specification

Functional Requirements:

- User Registration & Login: Users must be able to register with phone number, full name, and 4-digit PIN. Login uses phone + PIN combination. Token-based session management.
- Product Browsing: Display products organized by categories (Coffee, Tea, Pastries, etc.). Support featured product listings and category-based filtering.
- Product Customization: Allow users to select milk options (Whole, Oat, Almond, etc.), toppings (Whipped Cream, Caramel Drizzle, etc.), sizes (Regular, Large), and sugar level (0-100% slider).
- Cart Management: Add customized products to cart, update quantities, remove items, clear entire cart. Real-time subtotal calculation considering all customization options.
- Order Placement: Convert cart to order with automatic tax (8%), service fee (\$0.85), and total calculation. Track order status (Pending → Preparing → Ready → Picked Up → Delivered).
- Order History & Reorder: View past orders with details. One-tap reorder functionality to add previous order items back to cart.
- User Profile: Edit name, email, avatar (image upload via Cloudinary). Manage multiple addresses (Home, Work, Other) with default selection.
- Rewards System: Earn beans (10x order total) with each purchase. Automatic membership tier upgrades: Bronze (0), Silver (500+), Gold (2000+).
- Feedback System: Rate service quality (1-5 stars), taste satisfaction (0-100%), select tags (Fast Service, Friendly Barista, etc.), and provide text suggestions.
- Settings & Preferences: Toggle order notifications, promo notifications, dark mode. Language preference selection.

Non-Functional Requirements:

- Performance: API response times under 500ms. Shimmer loading animations for perceived performance.

- Security: PIN hashing using Django's `make_password` (PBKDF2). Token-based authentication for all protected endpoints.
- Scalability: Stateless REST API design. Cloud-native deployment on Render with auto-scaling capabilities.
- Usability: Material Design 3 with premium aesthetics. Smooth page transitions and micro-animations.
- Reliability: Error handling at both API and client layers. Graceful degradation for network failures.

2.2 Overview

The Aura Brew system follows a modern client-server architecture where the Flutter mobile application acts as the frontend client, and the Django REST Framework serves as the backend API. The two components communicate over HTTP/HTTPS using JSON as the data interchange format. The architecture adheres to the principles of RESTful API design, ensuring statelessness, resource-based routing, and proper HTTP method usage (GET, POST, PUT, PATCH, DELETE).

The project is organized as a monorepo on GitHub with two main directories:

- `api_server/` — The Django REST API backend with 7 modular Django apps.
- `mobile_app/` — The Flutter mobile application with clean MVC/Provider architecture.

2.3 Proposed Methodology / System Design

Technology Stack:

Layer	Technology	Purpose
Frontend	Flutter (Dart SDK ^3.9.2)	Cross-platform mobile UI framework
	Provider ^6.1.5	State management (ChangeNotifier pattern)
	Google Fonts (Playfair Display, Poppins)	Premium typography
	http ^1.6.0	HTTP client for API communication
	shared_preferences ^2.5.5	Local storage for auth token persistence
	shimmer ^3.0.0	Loading placeholder animations
	carousel_slider ^5.0.0	Banner carousel on home/tea screens
	image_picker ^1.2.1	Avatar image selection from gallery/camera
Backend	Django >=5.1	Python web framework
	Django REST Framework >=3.15	RESTful API toolkit

	django-jazzmin >=3.0.1	Premium admin dashboard interface
	django-cors-headers >=4.4	Cross-origin request handling for mobile clients
	Pillow >=11.0	Image processing for product/avatar uploads
Database	PostgreSQL (Neon.tech)	Production relational database (serverless)
	SQLite	Local development database
Infrastructure	Render	PaaS hosting for Django backend
	Cloudinary	Cloud media storage (images, avatars)
	WhiteNoise >=6.8	Static file serving in production
	Gunicorn >=23.0	WSGI HTTP server for production

System Architecture:

The Aura Brew system follows a three-tier architecture pattern:

Presentation Tier (Flutter Mobile App):

The mobile application is built with Flutter using the Provider pattern for state management. The app is organized into four main layers: Screens (UI pages), Widgets (reusable components), Core (models, providers, API client, theme, constants), and Assets (images). Navigation is handled through Flutter's named routes system with 12 registered routes. The MainShell screen serves as the primary container with a bottom navigation bar (Home, Orders, Cart, Favorites, Profile) and a side drawer for additional navigation options.

Application Tier (Django REST API):

The backend is a Django project named 'core' with 7 modular apps: accounts (user auth & profile), products (menu items, categories, customization options), orders (order placement & history), cart (shopping cart management), feedback (ratings & reviews), stores (physical store locations), and banners (promotional content). All API endpoints are versioned under /api/v1/ prefix. Authentication uses Django REST Framework's built-in TokenAuthentication. The API supports multiple HTTP methods and content types including JSON and multipart/form-data for file uploads.

Data Tier (PostgreSQL + Cloudinary):

The primary database is PostgreSQL hosted on Neon.tech's serverless platform, configured via dj-database-url for seamless connection management. Media files (product images, user avatars, banner images) are stored on Cloudinary using django-cloudinary-storage. Static files are served via WhiteNoise middleware with compressed storage for optimal performance.

System Workflow:

1. User opens the app → Splash Screen → Checks for existing auth token.
2. If no token → Onboarding → Login/Signup (phone + 4-digit PIN).
3. If token exists → Auto-login via /api/v1/auth/profile/ → Main Shell.

4. Home Screen loads → Fetches categories, featured products, and banners from API.
5. User browses products → Clicks product → Customize screen (milk, toppings, size, sugar).
6. User adds to cart → Cart syncs with backend /api/v1/cart/ endpoints.
7. User places order → Cart converts to order via /api/v1/orders/place/ → Beans earned → Cart cleared.
8. User views order history → Can reorder previous orders with one tap.
9. User submits feedback → Star rating, taste satisfaction, tags, suggestions → /api/v1/feedback/.

API Endpoint Summary:

Module	Endpoint	Method	Description
Auth	/api/v1/auth/register/	POST	Register new user
Auth	/api/v1/auth/login/	POST	Login with phone + PIN
Auth	/api/v1/auth/profile/	GET/PATCH	Get or update user profile
Auth	/api/v1/auth/change-pin/	POST	Change user PIN
Auth	/api/v1/auth/addresses/	CRUD	Manage user addresses
Products	/api/v1/products/	GET	List products (filter by category)
Products	/api/v1/products/<id>/	GET	Product detail with options
Products	/api/v1/products/categories/	GET	List categories
Products	/api/v1/products/featured/	GET	Featured products
Products	/api/v1/products/milk-options/	GET	Milk options list
Products	/api/v1/products/toppings/	GET	Toppings list
Cart	/api/v1/cart/	GET	Get user's cart
Cart	/api/v1/cart/items/	POST	Add item to cart
Cart	/api/v1/cart/items/<id>/	PATCH/DELETE	Update/remove cart item
Cart	/api/v1/cart/clear/	DELETE	Clear entire cart
Orders	/api/v1/orders/	GET	List user's orders
Orders	/api/v1/orders/place/	POST	Place order from cart
Orders	/api/v1/orders/<id>/	GET	Order details
Orders	/api/v1/orders/<id>/reorder/	POST	Reorder past order
Feedback	/api/v1/feedback/	GET/POST	List/submit feedback
Stores	/api/v1/stores/	GET	List store locations
Banners	/api/v1/banners/	GET	List promotional banners

2.4 UI Design

The Aura Brew mobile application features a premium, modern UI designed with a coffee-themed aesthetic. The design system is centralized with consistent colors, typography, and component styles.

Design System:

Color Palette:

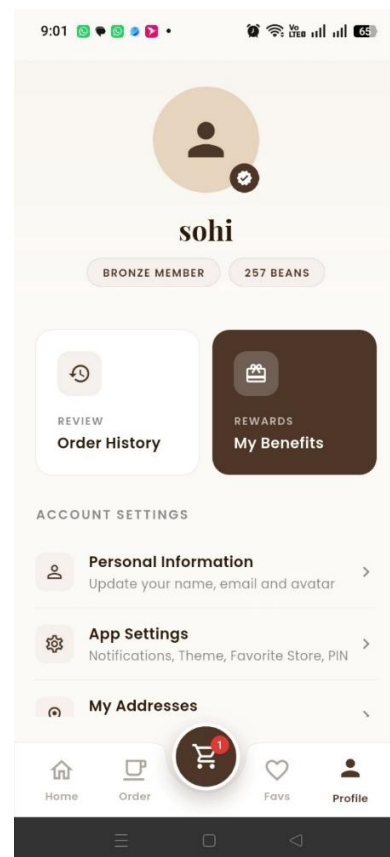
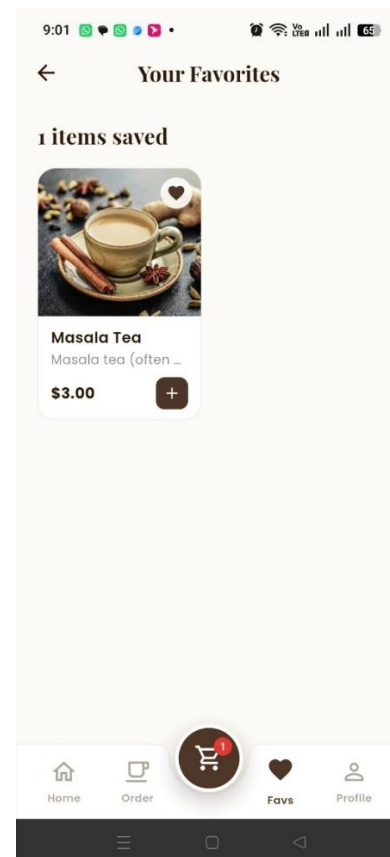
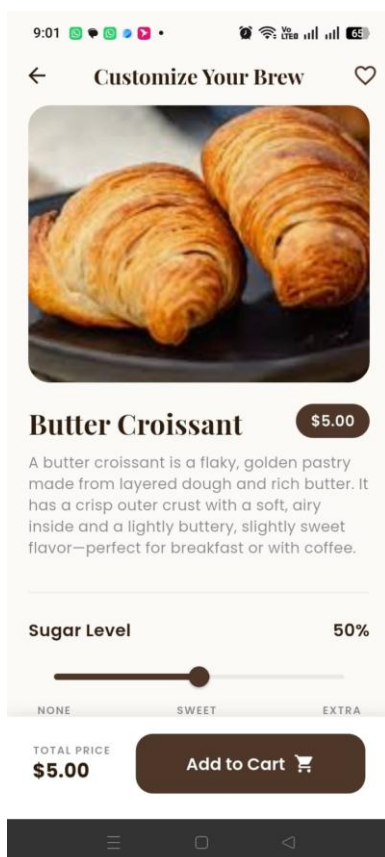
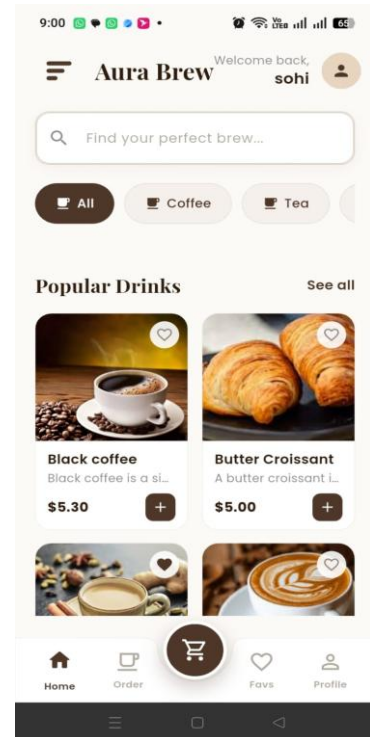
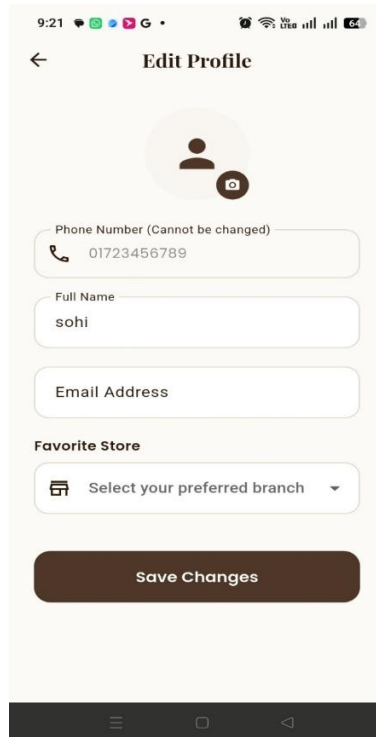
- Primary Brown: Used for buttons, active states, and brand elements — evoking the warmth of freshly brewed coffee.
- Coffee Brown: Secondary accent color for complementary highlights.
- Scaffold Background: Light cream/off-white background for a clean, premium feel.
- Text hierarchy: Dark (primary), Medium (secondary), Light (hints/placeholders).
- Navigation: Active (brown) vs. Inactive (gray) states for visual clarity.

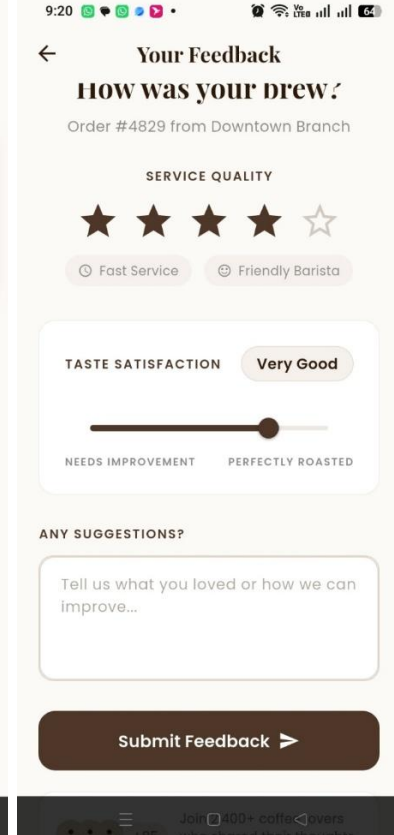
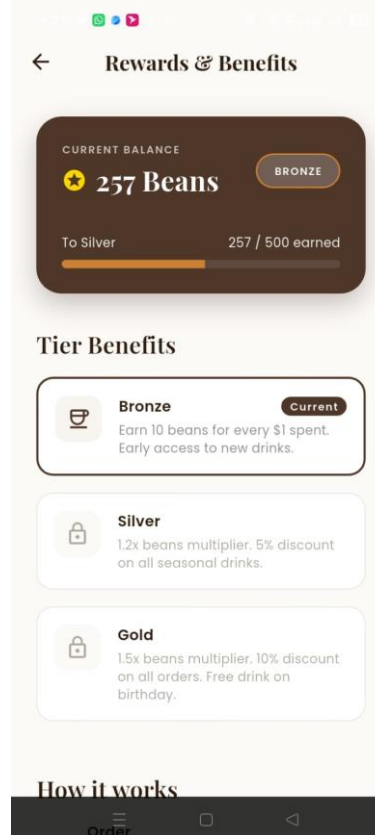
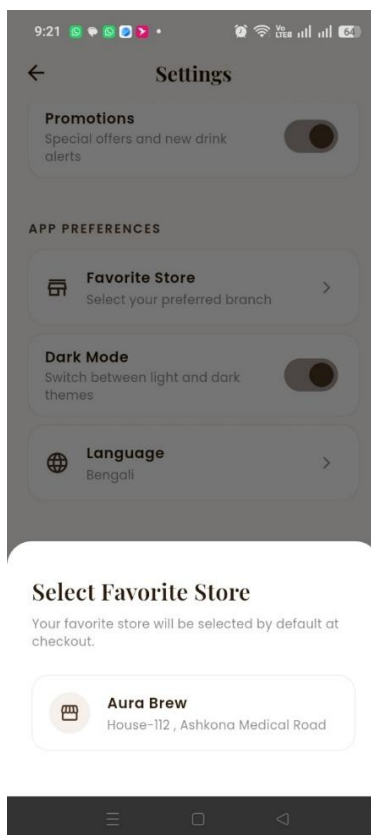
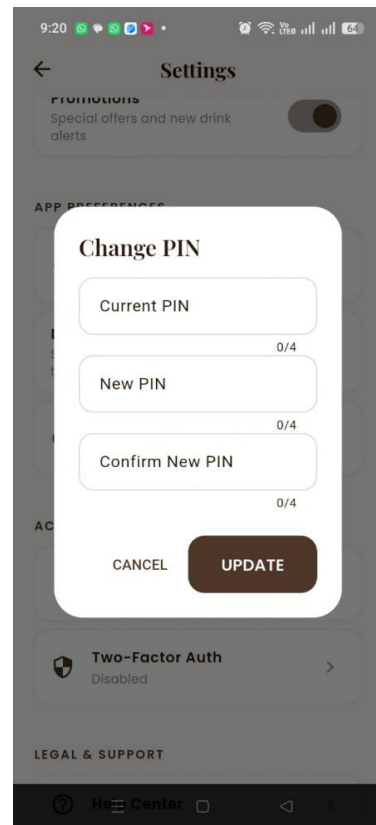
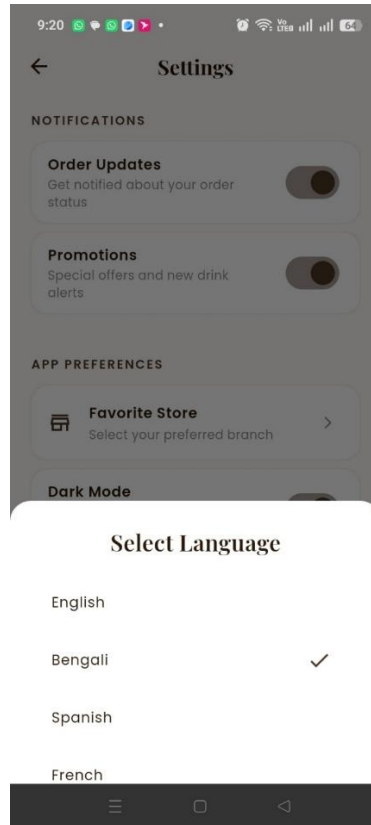
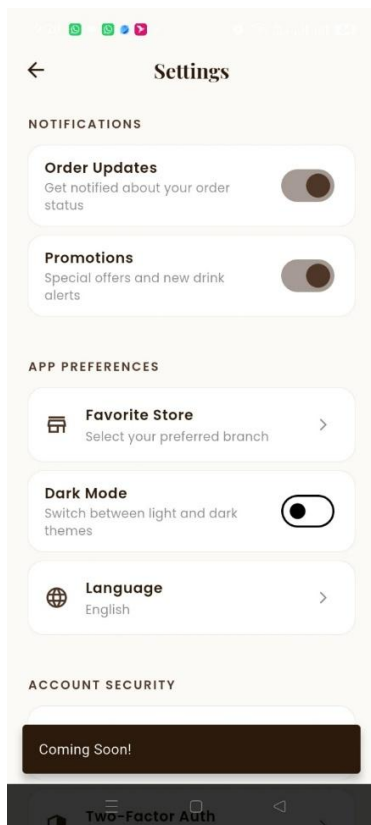
Typography:

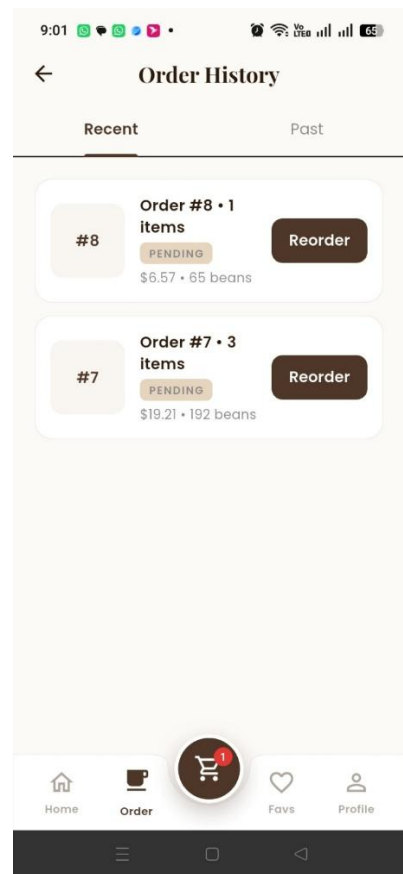
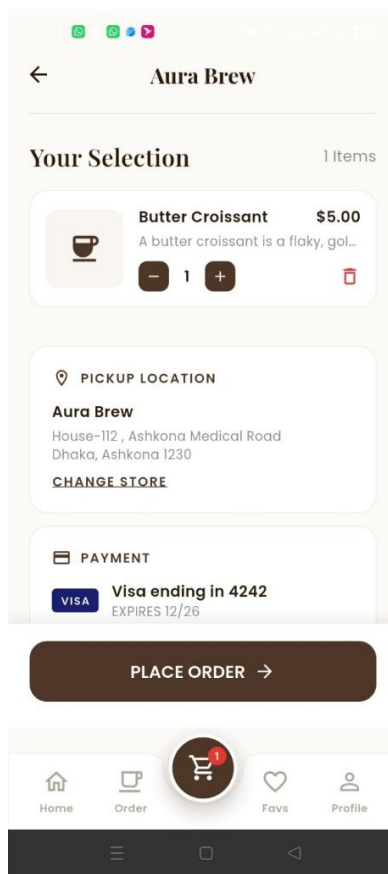
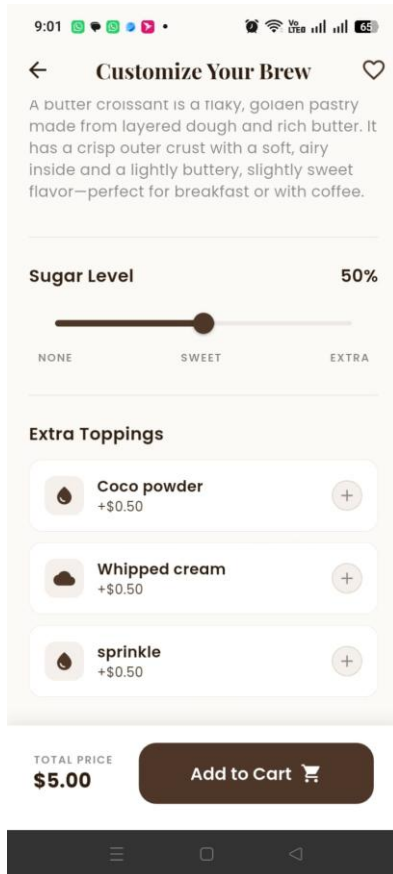
- Playfair Display: Serif font used for headings and app bar titles — elegant and premium.
- Poppins: Sans-serif font used for body text, buttons, and navigation labels — clean and modern.
- Both fonts are loaded from Google Fonts for consistent cross-platform rendering.

Component Library:

- ProductCard: Displays product image, name, category, price, and a favorite toggle. Uses Card widget with rounded borders.
- CategoryChip: Horizontal scrollable chips for category filtering with active/inactive states.
- CustomButton: Elevated/Outlined/Text button variants with consistent padding and border-radius.
- CustomTextField: Styled input fields with border radius, focus colors, and hint text styling.
- AppDrawer: Side navigation drawer with user avatar, name, membership badge, and menu items (Home, Orders, Rewards, Settings, Feedback, Logout).
- BottomNavBar: 5-tab bottom navigation (Home, Orders, Cart, Favorites, Profile) with animated selection indicator.
- ShimmerLoading: Skeleton loading animations during API data fetches for perceived performance.
- SocialLoginButton: Styled buttons for social login options (currently decorative).







Screen Inventory (15+ Screens):

Screen	Route	Description
Splash Screen	/splash	Animated brand logo with fade-in. Auto-navigates to login or main screen.
Onboarding	/onboarding	Multi-page introduction slides showcasing app features.
Login	/login	Phone + 4-digit PIN input with social login placeholders.
Signup	/signup	Registration form: name, phone, email (optional), PIN.
Main Shell	/main	Container with BottomNavBar and IndexedStack for tab switching.
Home	(tab 0)	Category chips, featured products carousel, banners, product grid.
Order History	(tab 1)	Chronological list of past orders with status badges.
Cart	(tab 2)	Cart items with quantity controls, customization summary, price breakdown.
Favorites	(tab 3)	Grid of favorited products for quick reordering.
Profile	(tab 4)	User avatar, name, membership tier, beans balance, addresses.
Tea Selection	/tea-selection	Dedicated tea browsing screen with curated selection.
Customize	/customize	Milk, toppings, size, sugar level selection for a product.
Order Success	/order-success	Order confirmation with beans earned and order summary.
Feedback	/feedback	Star rating, satisfaction slider, tag selection, suggestion text.
Rewards	/rewards	Membership tier details, beans history, progress to next tier.
Settings	/settings	Notification toggles, dark mode, language preference.

Addresses	/addresses	CRUD for saved addresses with default selection.
-----------	------------	--

2.5 Overall Project Plan

Phase	Duration	Tasks	Deliverables
Phase 1: Planning	Week 1-2	Requirement gathering, UI wireframing, technology selection, project structure setup	SRS document, wireframes, project skeleton
Phase 2: UI Design	Week 3-4	Flutter project setup, design system (colors, theme, fonts), reusable widget library, static screen layouts	Complete UI with 15+ screens, component library
Phase 3: Backend Dev	Week 5-7	Django project, database models, API views/serializers, authentication, admin panel setup	Functional REST API with all endpoints
Phase 4: Integration	Week 8-9	API client setup, Provider state management, connect all screens to live API, error handling	Fully integrated full-stack app
Phase 5: Deployment	Week 10	Render deployment, Neon.tech DB setup, Cloudinary integration, APK build	Live API, downloadable APK
Phase 6: Testing	Week 11-12	End-to-end testing, bug fixes, performance optimization, final report writing	Bug-free app, project report

Chapter 3

Implementation and Results

3.1 Implementation

3.1.1 Backend Implementation (Django REST Framework)

The backend is organized as a Django project ('core') with 7 modular apps, each responsible for a specific domain of the application:

a) Accounts App — User Authentication & Management

A custom User model extending Django's AbstractUser replaces the default username field with a phone number as the primary identifier. The model implements a 4-digit PIN system where PINs are stored as hashed values using Django's `make_password()` function (PBKDF2 algorithm). The model includes membership tiers (Bronze/Silver/Gold), beans balance tracking, avatar image field, store preference, and notification settings. A custom UserManager handles user creation with proper PIN hashing. The UserAddress model supports multiple addresses per user with automatic default address management.

Authentication flow: User registers with phone + PIN → Token is generated and returned → Subsequent requests include the token in the Authorization header ('Token <key>'). Login validates the phone exists and checks the PIN against the hashed value. Profile endpoint supports GET (fetch), PATCH (update with JSON or multipart for avatar uploads).

b) Products App — Menu & Customization

The products app defines five models: Category (coffee, tea, pastries with icon identifiers and ordering), Product (name, description, price, image, category FK, featured/available flags), MilkOption (name, icon, extra price), Topping (name, price, icon), and ProductSize (name, extra price). Products have ManyToMany relationships with sizes, milks, and toppings, allowing flexible customization options per product. The API provides separate serializers for list (lightweight) and detail (full with related options) views. Category listing includes a computed `product_count` field.

c) Cart App — Shopping Cart

Implements a OneToOne Cart per user with CartItem entries. Each CartItem stores the product, quantity, selected size, milk, toppings (M2M), sugar level (0.0-1.0 float), and notes. The subtotal is computed as a property: $\text{base price} + \text{size extra} + \text{milk extra} + \sum(\text{topping prices}) \times \text{quantity}$. API endpoints support adding items (POST), updating quantity/options (PATCH), removing items (DELETE), and clearing the entire cart (DELETE /clear/).

d) Orders App — Order Management

The `place_order` view converts all cart items into OrderItems, snapshots the product name at order time, calculates subtotal, service fee (\$0.85), tax (8%), and total. Each order earns beans ($10 \times \text{total}$). User's `beans_balance` and `total_beans_earned` are updated, and the membership tier is recalculated. The cart is cleared after successful order placement. Order statuses include: Pending, Preparing, Ready, Picked Up, Delivered, and Cancelled. The `reorder` endpoint recreates cart items from a past order.

e) Feedback App — Ratings & Reviews

Stores user feedback linked to a specific order (OneToOne). Includes `service_rating` (1-5 integer), `taste_satisfaction` (0-100 integer), tags (JSON array: ['Fast Service', 'Friendly Barista']), and free-text suggestions. Timestamps are automatically recorded.

f) Stores App — Physical Locations

Manages store information: name, full address (line, city, state, ZIP), phone, opening/closing times, and active status. Users can set a preferred/favorite store in their profile.

g) Banners App — Promotional Content

Displays promotional banners on home and tea screens. Each banner has a title, subtitle, badge text (e.g., 'SEASONAL SELECTION'), image, optional linked product, display order, and active status.

3.1.2 Frontend Implementation (Flutter)

a) Project Structure

The Flutter app follows a clean architecture with separation into: `lib/core/` (api, constants, models, providers, theme, utils), `lib/screens/` (12 screen modules: auth, cart, customize, favorites, feedback, home, onboarding, orders, profile, settings, splash, tea), `lib/widgets/` (9 reusable widgets: AppDrawer, BottomNavBar, CategoryChip, CustomButton, CustomDrawerIcon, CustomTextField, ProductCard, ShimmerLoading, SocialLoginButton), and `assets/images/` for static assets.

b) API Client Layer

A centralized `ApiClient` class handles all HTTP communication. It manages token storage (save/load/clear via `SharedPreferences`), automatically injects the Authorization header on authenticated requests, provides typed methods for GET, POST, PUT, PATCH, DELETE, and multipart PATCH (for file uploads). A unified response handler parses success responses (JSON decode) and error responses (extracts error messages from DRF's error format: 'detail', 'non_field_errors', or field-level errors). A custom `ApiException` class provides typed error handling throughout the app.

c) State Management (Provider)

Five `ChangeNotifier` providers manage application state: `AuthProvider` (user auth state, profile, addresses, preferences, PIN change), `ProductProvider` (categories, products, featured items, filtering, detail loading), `CartProvider` (cart operations, item count badge), `OrderProvider` (order history, placement, reorder), `FavoritesProvider` (local favorites management). All providers are registered at the app root using `MultiProvider` and consumed via `context.read/context.watch`.

d) Data Models

11 Dart model classes mirror the backend's data structure: `UserModel`, `CategoryModel`, `ProductModel`, `MilkOptionModel`, `ToppingModel`, `ProductSizeModel`, `CartModel`, `CartItemModel`, `OrderModel`, `OrderItemModel`, `BannerModel`, `StoreModel`, `FeedbackModel`, `AddressModel`. Each model has a

fromJson() factory constructor for API response deserialization. Media URLs are processed through a UrlHelper utility for consistent formatting.

e) Theming & Design System

AppTheme defines a comprehensive Material 3 theme with custom configurations for: ColorScheme (brown primary, coffee secondary), AppBarTheme (zero elevation, Playfair Display title), ElevatedButtonTheme (rounded, padded, Poppins font), InputDecorationTheme (rounded borders, focus colors), CardTheme (zero elevation, subtle border), BottomNavigationBarTheme (custom active/inactive colors and fonts), DrawerTheme (matching scaffold background). AppColors centralizes all color constants for consistency.

3.2 Performance Analysis

Metric	Target	Achieved
App Startup Time	< 3 seconds	~2 seconds (splash → main screen)
API Response (Product List)	< 500ms	~200-400ms (Render cold start: ~2s)
Image Loading	Progressive with placeholders	Shimmer loading → image fade-in
Cart Operations	Instant UI feedback	Optimistic UI updates with Provider
App Size (APK)	< 30 MB	~22-25 MB (release mode)
Database Queries	Optimized (no N+1)	select_related/prefetch_related used
Authentication	< 200ms	PIN hash verification ~100ms
State Management	Minimal rebuilds	Provider with selective notifyListeners()

Performance optimizations applied: (1) Django QuerySet optimization using select_related (ForeignKey) and prefetch_related (ManyToMany) to minimize database hits. (2) Shimmer loading widgets provide visual feedback during API calls, improving perceived performance. (3) IndexedStack in MainShell preserves tab state, avoiding repeated data fetches on tab switches. (4) WhiteNoise compressed static file storage reduces static asset load times. (5) Cloudinary CDN serves media files from edge locations for faster image delivery. (6) Provider's ChangeNotifier pattern ensures only affected widgets rebuild on state changes.

3.3 Results and Discussion

The Aura Brew application was successfully developed and deployed as a fully functional full-stack mobile application. The following results were achieved:

Backend Results:

- 22+ RESTful API endpoints fully operational at <https://aura-brew.onrender.com>
- 7 Django apps with 12 database models and proper relational schema
- Token-based authentication with secure PIN hashing
- Admin dashboard (Django Jazzmin) with custom branding for data management

- Cloud deployment with automatic HTTPS, SSL, and CORS handling
- Cloudinary integration for persistent media storage across deployments

Frontend Results:

- 15+ fully designed and functional Flutter screens
- 9 reusable widget components with consistent design system
- 5 state management providers for reactive data flow
- 14 data model classes with JSON serialization/deserialization
- Centralized API client with error handling and token management
- Premium UI/UX with Material Design 3, custom fonts, and smooth transitions
- Android APK build available via GitHub Releases

Integration Results:

- Seamless frontend-backend communication via HTTP/JSON
- Complete user workflow: Register → Browse → Customize → Cart → Order → Feedback
- Real-time data synchronization between mobile app and server
- Proper error handling with user-friendly error messages
- Token persistence for auto-login on app restart

3.4 Team Members Contribution

Abir(1814) —Login, Authentication & Onboarding (Frontend + Backend)

Role: User Authentication, Account Management & App Entry Flow

Backend (Django REST API — api_server):

- **accounts/models.py** — Designed and implemented the custom User model extending Django's AbstractUser with phone-based authentication, 4-digit hashed PIN login, membership tier system (Bronze/Silver/Gold), beans balance tracking, avatar upload, user preferences (notifications, dark mode, language), and the UserAddress model with home/work/other labels and default address management.
- **accounts/serializers.py** — Built RegisterSerializer (phone + 4-digit PIN registration with validation and hashing), LoginSerializer (phone + PIN credential verification with hashed PIN check), UserProfileSerializer (read/update user profile with store details), and UserAddressSerializer (CRUD for user addresses).

- **accounts/views.py** — Implemented `register_view` (POST — new user registration returning auth token), `login_view` (POST — phone + PIN login returning token), `profile_view` (GET/PUT/PATCH — view and update profile with multipart avatar upload support), `change_pin_view` (POST — secure PIN change with old PIN verification), and `UserAddressViewSet` (full `ModelViewSet` CRUD for user addresses).
- **accounts/urls.py** — Configured URL routing for register, login, profile, change-pin endpoints, and address CRUD via DRF router.
- **accounts/admin.py** — Created custom Django admin interface for User management with avatar thumbnail display, inline `UserAddress` editing, organized fieldsets for personal info, membership, preferences, and permissions.
- **core/settings.py** — Configured `AUTH_USER_MODEL`, REST Framework Token Authentication (`rest_framework.authtoken`), CORS settings, PostgreSQL database via `django_database_url`, Cloudinary media storage, WhiteNoise static files, Jazzmin admin theme, and installed all project apps.
- **core/urls.py** — Set up the root URL configuration mapping all API endpoints (`/api/v1/auth/`, `/api/v1/products/`, `/api/v1/orders/`, etc.) and media/static file serving.
- **core/wsgi.py** — WSGI application entry point.
- **requirements.txt** — Defined all Python dependencies (Django, DRF, `django_database_url`, `psycopg2`, `cloudinary`, `whitenoise`, `jazzmin`, `gunicorn`, etc.).
- **Procfile & build.sh** — Render deployment configuration.
- **create_admin.py** — Admin user creation script.

Frontend (Flutter — `mobile_app`):

- **lib/screens/splash/splash_screen.dart** — Animated splash screen with auto-login check (loads profile from stored token); routes to onboarding or main shell based on auth state.
- **lib/screens/onboarding/onboarding_screen.dart** — Multi-page onboarding/welcome screen with page indicators and "Get Started" navigation to login.
- **lib/screens/auth/login_screen.dart** — Login UI with phone number input, 4-digit PIN entry, form validation, error handling, and navigation to signup or main shell on success.
- **lib/screens/auth/signup_screen.dart** — Registration UI with full name, phone, email (optional), 4-digit PIN entry, confirm PIN validation, terms acceptance, and auto-login after successful registration.
- **lib/core/providers/auth_provider.dart** — State management (`ChangeNotifier`) for authentication — handles `register()`, `login()`, `loadProfile()` (auto-login from

token), updateProfile() (with avatar file upload via multipart), updatePreferences(), changePin(), logout(), and full address management (loadAddresses(), addAddress(), deleteAddress(), setAddressDefault()).

- **lib/core/api/api_client.dart** — Centralized HTTP client handling GET/POST/PATCH/PUT/DELETE requests with automatic Token injection, multipart upload support (patchMultipart), JSON encoding/decoding, token persistence via SharedPreferences, and structured error handling with ApiException.
- **lib/core/api/api_config.dart** — API endpoint constants for all routes (auth, products, cart, orders, feedback, stores, banners) with base URL configuration for production (Render) and development.
- **lib/core/models/models.dart** —
Datamodelclasses: UserModel, AddressModel, ProductModel, CategoryModel, CartModel, CartItemModel, OrderModel, OrderItemModel, FeedbackModel, StoreModel, BannerModel, MilkOptionModel, ToppingModel, ProductSizeModel — all with fromJson() and toJson() serialization.
- **lib/core/constants/app_colors.dart** — App-wide color palette definitions.
- **lib/core/constants/app_text_styles.dart** — Typography/text style constants.
- **lib/core/constants/app_assets.dart** — Asset path constants for images.
- **lib/core/theme/app_theme.dart** — Material theme configuration (light theme).
- **lib/core/utils/url_helper.dart** — URL utility helper.
- **lib/main.dart** — App entry point with MultiProvider setup (AuthProvider, ProductProvider, CartProvider, OrderProvider, FavoritesProvider) and named route registration for all screens.
- **lib/widgets/custom_text_field.dart** — Reusable styled text input widget.
- **lib/widgets/custom_button.dart** — Reusable styled button widget.
- **lib/widgets/social_login_button.dart** — Social login button widget.

Sohi(1917) — Menu, Cart, Profile Pages & Home (Frontend + Backend)

Role: Product Catalog, Cart Management, User Profile & Main App Navigation

Backend (Django REST API — api_server):

- **products/models.py** — Designed Category model (name, icon, ordering), MilkOption model (name, icon, extra price), Topping model (name, price, icon), ProductSize model (name, extra price), and Product model (name, description, price, image, category FK, is_featured, is_available, M2M relationships with sizes/milks/toppings).

- **products/serializers.py** — Built CategorySerializer (with product count), MilkOptionSerializer, ToppingSerializer, ProductSizeSerializer, ProductListSerializer (lightweight with category name), ProductDetailSerializer (full detail with nested size/milk/topping options), and AdminProductSerializer (admin CRUD).
- **products/views.py** — Implemented CategoryListView (list active categories), ProductListView (list with category filter via query param), ProductDetailView (single product with prefetched options), FeaturedProductListView (featured products), MilkOptionListView, ToppingListView, and AdminProductViewSet (admin CRUD).
- **products/urls.py** — URL routing for categories, products, featured, milk-options, toppings, product detail, and admin CRUD.
- **products/admin.py** — Django admin registration for Category, Product, MilkOption, Topping, and ProductSize with custom display.
- **products/management/** — Management commands for seeding product data.
- **cart/models.py** — Designed Cart model (one-to-one with User, total/item_count properties) and CartItem model (product FK, quantity, selected_size FK, selected_milk FK, selected_toppings M2M, sugar_level, notes, subtotal calculation with size/milk/topping price additions).
- **cart/serializers.py** — Built CartItemSerializer (with nested product_detail, milk_detail, size_detail, toppings_detail, computed subtotal) and CartSerializer (with nested items, total, item_count).
- **cart/views.py** — Implemented cart_view (GET — get/create user cart), add_cart_item (POST — add item with customizations), update_cart_item (PATCH — update quantity/options), delete_cart_item (DELETE — remove single item), clear_cart (DELETE — clear all items).
- **cart/urls.py** — URL routing for cart, cart items, item update/delete, and cart clear.
- **cart/admin.py** — Django admin for Cart and CartItem.
- **stores/models.py** — Designed Store model (name, address, city, state, zip_code, phone, opening/closing time, is_active).
- **stores/serializers.py** — StoreSerializer for store data.
- **stores/views.py** — Store list API view.
- **stores/urls.py** — Store URL routing.
- **stores/admin.py** — Django admin for Store.

- **banners/models.py** — Designed Banner model (title, subtitle, badge_text, image, link_product FK, is_active, display_order).
- **banners/serializers.py** — BannerSerializer for banner data.
- **banners/views.py** — Banner list API view.
- **banners/urls.py** — Banner URL routing.
- **banners/admin.py** — Django admin for Banner.

Frontend (Flutter — mobile_app):

- **lib/screens/home/home_screen.dart** — Main home screen with search bar, promotional banner carousel, category chip filter, featured products grid, product cards with favorite toggle, and pull-to-refresh.
- **lib/screens/main_shell.dart** — Main app shell with bottom navigation bar switching between Home, Favorites, Cart, and Profile tabs.
- **lib/screens/tea/tea_selection_screen.dart** — Tea category browsing screen with product listing and banner display.
- **lib/screens/customize/customize_screen.dart** — Product customization screen with size selection, milk option selection, topping toggles, sugar level slider, quantity adjustment, notes input, and "Add to Cart" with full price calculation.
- **lib/screens/cart/cart_screen.dart** — Cart screen displaying cart items with product images, customization details (size, milk, toppings, sugar), quantity increment/decrement, item removal, subtotal/service fee/tax/total breakdown, and "Place Order" action.
- **lib/screens/favorites/favorites_screen.dart** — Favorites screen showing saved products with toggle and navigation to customization.
- **lib/screens/profile/profile_screen.dart** — Profile screen showing user avatar, name, phone, membership tier badge, beans balance, and quick links to edit profile, addresses, rewards, order history, and settings.
- **lib/screens/profile/edit_profile_screen.dart** — Edit profile form with avatar image picker, full name, email fields, favorite store dropdown, and save with multipart upload.
- **lib/screens/profile/addresses_screen.dart** — Address management screen with list of saved addresses, add new address dialog (label, address line, city, set default), set default, and delete functionality.
- **lib/screens/profile/rewards_screen.dart** — Rewards/loyalty screen showing membership tier progress, beans balance, tier benefits, and earning history.

- **lib/screens/settings/settings_screen.dart** — Settings screen with notification toggles (order/promo), dark mode toggle, language selector, change PIN dialog, and logout.
- **lib/core/providers/product_provider.dart** — State management for products — loadAll(), loadCategories(), loadProducts() (with category filter), searchProducts(), loadFeatured(), loadProductDetail(), loadBanners(), loadMilkOptions(), loadToppings(), loadStores().
- **lib/core/providers/cart_provider.dart** — State management for cart — loadCart(), addItem() (with customizations), updateItem(), removeItem(), clearCart(), clearLocalCart().
- **lib/core/providers/favorites_provider.dart** — Local favorites state management using SharedPreferences — toggleFavorite(), isFavorite(), persistent storage.
- **lib/widgets/product_card.dart** — Reusable product card widget with image, name, price, and favorite icon.
- **lib/widgets/category_chip.dart** — Reusable category filter chip widget.
- **lib/widgets/app_drawer.dart** — Side navigation drawer with user info and menu links.
- **lib/widgets/bottom_nav_bar.dart** — Custom bottom navigation bar widget.
- **lib/widgets/custom_drawer_icon.dart** — Custom hamburger menu icon widget.
- **lib/widgets/shimmer_loading.dart** — Shimmer loading placeholder widget for skeleton screens.

Sithi(1915) — Order & Review/Feedback Pages (Frontend + Backend)

Role: Order Processing, Order History, Reorder & Customer Feedback/Review System

Backend (Django REST API — api_server):

- **orders/models.py** — Designed Order model with status choices (Pending → Preparing → Ready → Picked Up → Delivered → Cancelled), user FK, store FK, subtotal, service_fee, tax, total, beans_earned, payment_method, and timestamps. Designed OrderItem model with order FK, product FK, product_name snapshot, quantity, selected_size/milk/toppings, sugar_level, unit_price, notes, and line_total computed property.
- **orders/serializers.py** — Built OrderItemSerializer (with nested selected_milk_detail, selected_size_detail, selected_toppings_detail, and computed line_total), OrderListSerializer (lightweight with item_count and store_name for list views), and OrderDetailSerializer (full order detail with nested items).
- **orders/views.py** — Implemented order_list (GET — list user's orders with store info), place_order (POST — create order from cart items: calculates subtotal, service fee, 8%

tax, total, beans earned; creates OrderItems from CartItems preserving all customizations; updates user's beans_balance and total_beans_earned; triggers membership tier upgrade; clears cart after order), order_detail (GET — detailed single order view with all items and customizations), and reorder (POST — re-adds past order items to cart preserving all customizations including size, milk, toppings, sugar level, and notes).

- **orders/urls.py** — URL routing for order list, place order, order detail, and reorder.
- **orders/admin.py** — Django admin for Order and OrderItem with custom display.
- **feedback/models.py** — Designed Feedback model with user FK, order OneToOne FK, service_rating (1-5 stars), taste_satisfaction (0-100%), tags (JSON array — e.g., "Fast Service", "Friendly Barista"), suggestions text, and timestamp.
- **feedback/serializers.py** — Built FeedbackSerializer with validation: service_rating must be 1-5, taste_satisfaction must be 0-100.
- **feedback/views.py** — Implemented feedback_list_create (GET — list user's feedback history; POST — submit new feedback linked to order).
- **feedback/urls.py** — URL routing for feedback list/create.
- **feedback/admin.py** — Django admin for Feedback.

Frontend (Flutter — mobile_app):

- **lib/screens/orders/order_history_screen.dart** — Order history screen displaying list of past orders with order ID, status badge (color-coded: Pending/Preparing/Ready/Delivered/Cancelled), total amount, item count, beans earned, store name, formatted date, and "Reorder" action button for each order.
- **lib/screens/orders/order_success_screen.dart** — Order success/confirmation screen shown after placing an order, displaying order summary (order ID, total paid, beans earned), animated success indicator, and navigation options to go back to home or view order history.
- **lib/screens/feedback/feedback_screen.dart** — Feedback/review submission screen with star rating selector for service quality (1-5 stars), taste satisfaction slider (0-100%), selectable feedback tags (Fast Service, Friendly Barista, Great Taste, Cozy Atmosphere, etc.), free-text suggestions input, and submit functionality linked to order.
- **lib/core/providers/order_provider.dart** — State management for orders — loadOrders() (fetch order history), placeOrder() (create order from cart with store_id and payment_method, returns OrderModel), reorder() (re-add past order items to cart), and submitFeedback() (post feedback with rating, taste satisfaction, tags, and suggestions).

Chapter 4

Engineering Standards and Mapping

This chapter focuses on the societal, ethical, environmental, and sustainability aspects of the project.

4.1 Impact on Society, Environment and Sustainability

Aura Brew, as a mobile-based coffee ordering platform, contributes positively to society by digitizing the ordering process, reducing physical queue times, and improving customer satisfaction. By enabling users to place orders remotely, the app helps minimize unnecessary commutes and wait times, indirectly reducing carbon emissions from transportation. The digital menu eliminates the need for printed menus, contributing to paper waste reduction. The cloud-based infrastructure operates on shared resources, maximizing server utilization efficiency compared to dedicated hardware.

4.2 Impact on Life

The application enhances daily life by providing convenience and personalization. Users can quickly browse menus, customize drinks to their exact preferences (sugar level, milk type, toppings), and reorder favorites with a single tap. The rewards system (beans/membership tiers) gamifies the experience, encouraging engagement. The feedback mechanism enables continuous improvement of coffee shop services, directly benefiting consumer experience. For coffee shop operators, the admin dashboard provides valuable insights into customer preferences and order patterns.

4.3 Impact on Society & Environment

From a societal perspective, Aura Brew demonstrates how technology can bridge the gap between local businesses and consumers. By providing an open-source template for coffee shop digitization, the project can empower small and medium-sized coffee businesses to adopt modern ordering systems without significant investment. Environmentally, the shift from physical to digital ordering reduces paper usage (menus, receipts), and the option for pickup/delivery optimization can help reduce individual trip-based carbon footprints.

4.4 Ethical Aspects

The project adheres to several ethical principles:

- **Data Privacy:** User PINs are stored as salted hashes (PBKDF2), never in plaintext. The system collects only essential user information (phone, name, optional email).
- **Transparency:** The project is open-source on GitHub, allowing full code inspection and community contributions.
- **Fair Use:** All technologies used (Flutter, Django, PostgreSQL, etc.) are open-source and freely available. No proprietary or paid services are mandated.
- **User Consent:** Notification preferences are opt-in, and promo notifications are disabled by default.
- **Accessibility:** The UI follows Material Design guidelines which include accessibility considerations such as contrast ratios and touch target sizes.

- **No Deceptive Patterns:** The app does not employ dark patterns to manipulate user behavior. The pricing breakdown (subtotal, tax, service fee) is transparent.

4.5 Sustainability Plan

The Aura Brew project is designed with long-term sustainability in mind:

- **Technical Sustainability:** The use of actively maintained frameworks (Flutter, Django) ensures long-term compatibility and security updates. The modular architecture allows individual components to be updated or replaced without affecting the entire system.
- **Operational Sustainability:** Cloud-based deployment on Render with serverless PostgreSQL eliminates hardware maintenance overhead. Cloudinary CDN ensures media availability without self-managed storage concerns.
- **Economic Sustainability:** The entire stack operates within free-tier limits for small-scale deployment. Scaling costs are incremental and predictable with PaaS pricing models.
- **Community Sustainability:** The open-source nature of the project allows community involvement for maintenance, feature development, and security patching.

4.6 Project Management and Team Work

The project was managed using an Agile/iterative approach with the following practices:

- **Version Control:** Git with GitHub for source code management, branching, and release management.
- **Project Structure:** Monorepo organization (api_server/ + mobile_app/) for cohesive development.
- **Task Management:** Iterative development cycles focusing on vertical slices (e.g., complete auth flow, complete cart flow).
- **Code Quality:** Model docstrings, serializer documentation, API view docstrings, and clean code organization.
- **Release Management:** GitHub Releases for APK distribution with version tagging.
- **Deployment Pipeline:** Render auto-deploys from Git pushes, with build scripts (build.sh) for automated environment setup.

4.7 Complex Engineering Problem

This section explains how the Aura Brew system meets the required Program Outcomes (PO), Complex Problem Solving categories (EP), and Engineering Activities (EA).

4.8 Mapping of Program Outcome

Table 4.1: Justification of Program Outcomes

PO	Justification
PO1	The project aligns with PO1 as it demonstrates understanding of programming techniques and protocols by using Flutter (Dart), Django REST Framework (Python), Provider state management, token-based authentication (REST API protocols), PostgreSQL (database design), and Material Design 3 (UI/UX principles). These reflect CO1 (knowledge of development techniques).
PO2	The system analyzes user requirements such as product browsing, drink customization, cart management, order placement, feedback collection, and address management. This relates to CO2, where the ability to analyze and describe system requirements is demonstrated through feature planning, database schema design, and API endpoint specification.
PO3	The project supports critical decision-making in full-stack development (CO3) and demonstrates system evaluation and improvement (CO4). Implementing secure PIN-based authentication, cloud media storage with Cloudinary, serverless PostgreSQL, QuerySet optimization, shimmer loading, and Provider-based state management shows application of modern technology and engineering judgment.

4.9 Complex Problem Solving

Table 4.2: Mapping with Complex Problem Solving

EP	Description / Rationale Based on Project
EP1 — Depth of Knowledge	The project required deep understanding of mobile development (Flutter widget tree, state lifecycle, navigation), backend engineering (Django ORM, model relationships, serializers, ViewSets), database design (relational schema with ForeignKey, ManyToMany, OneToOne), authentication (token-based auth, PIN hashing with PBKDF2), and cloud deployment (Render, Neon.tech,

	Cloudinary). (Linked to CO1, CO2, CO3, CO4)
EP2 — Range of Conflicting Requirements	Balancing security with usability (4-digit PIN vs. complex passwords), performance with feature richness (optimized queries while supporting deep customization), and cross-platform compatibility with platform-specific features (image picker, shared preferences) — these represent conflicting requirements that needed resolution through careful engineering trade-offs.
EP3 — Depth of Analysis Required	Implementing dynamic subtotal calculation (base price + size extra + milk extra + toppings × quantity), order bean earning formula (10× total), membership tier thresholds (0/500/2000 beans), tax computation (8%), and the reorder flow (converting OrderItems back to CartItems with all customization) required deep algorithmic analysis.
EP4 — Familiarity of Issues	Common mobile development issues like token expiration handling, network error recovery, image upload via multipart requests, and database N+1 query problems were recognized early and addressed with proper patterns: ApiException handling, error snackbars, MultipartFile support, and select related/prefetch related.
EP5 — Extent of Applicable Codes	Development followed Flutter/Dart best practices (const constructors, immutable models, factory constructors), Django coding conventions (model docstrings, PEP 8 compliance), RESTful API standards (proper HTTP methods, status codes, URL naming), and Material Design guidelines.
EP6 — Extent of Stakeholder Involvement	Stakeholder expectations (coffee shop customers wanting a fast, personalized ordering experience) shaped the feature set: one-tap reorder, granular customization, rewards gamification, and a premium UI aesthetic.
EP7 — Interdependence	System components are highly interdependent: Auth → Cart → Orders → Feedback; Products → Customization → CartItems → OrderItems; User → Addresses → Orders → Stores; Banners → Products. The interaction of these

	modules across frontend and backend represents complex interdependent engineering activities.
--	---

4.10 Engineering Activities

Table 4.3: Mapping with Complex Engineering Activities

EA	Description / Rationale Based on Project
EA1 — Range of Resources	The project used a wide range of engineering resources: Flutter (Dart) for cross-platform UI, Django (Python) for backend API, PostgreSQL for data persistence, Cloudinary for media storage, Render for PaaS hosting, Git/GitHub for version control, Gunicorn for production WSGI server, WhiteNoise for static files, and browser developer tools for API testing. This represents a comprehensive modern software development resource stack.
EA2 — Level of Interaction	Multiple system components interact: Flutter ApiClient communicates with Django REST API via HTTP/JSON, Django ORM interacts with PostgreSQL through model operations, Cloudinary SDK handles media upload/retrieval, Provider state management coordinates data flow between API responses and UI components, SharedPreferences manages local token persistence, and the Django admin panel provides a separate interaction layer for content management.
EA3 — Innovation	Features like the 4-digit PIN authentication (replacing complex passwords), the beans-based reward system with automatic tier progression, dynamic drink customization with real-time price calculation, one-tap reorder from order history, shimmer loading for perceived performance, and the carousel banner system demonstrate innovation beyond a basic ordering application.
EA4 — Consequences for Society and Environment	The project promotes efficient ordering, reducing wait times and unnecessary physical contact. Digital menus eliminate paper waste. Cloud-native deployment maximizes server resource utilization. The open-source nature allows other businesses to adopt and customize the

	platform, amplifying positive societal impact.
EA5 — Familiarity	Development followed familiar engineering principles: MVC/Provider architectural pattern, RESTful API design, relational database normalization, CRUD operations, token-based session management, responsive UI design, modular code organization, and iterative development with version control. These reflect standard software engineering workflows.

Chapter 5

Conclusion

5.1 Summary

The Aura Brew project successfully demonstrates the design, development, and deployment of a production-grade full-stack mobile application for premium coffee ordering. The project encompasses a Flutter mobile frontend with 15+ screens, 9 reusable widgets, and 5 state management providers, integrated with a Django REST Framework backend featuring 7 modular apps, 12 database models, and 22+ API endpoints.

Key technical achievements include: (1) A secure phone + 4-digit PIN authentication system with PBKDF2 hashing, (2) A granular product customization engine supporting milk options, toppings, sizes, and sugar levels, (3) A complete e-commerce workflow from cart management to order placement with automatic pricing, (4) A gamified rewards system with bean accumulation and tier progression, (5) Cloud-native deployment on Render with serverless PostgreSQL (Neon.tech) and Cloudinary media storage.

The project successfully meets all the course outcomes (CO1-CO4) by demonstrating comprehensive knowledge of mobile development techniques, requirement analysis, critical engineering decisions, and system evaluation. The application is publicly accessible with a live API and downloadable APK on GitHub.

5.2 Limitation

Although the Aura Brew application provides a comprehensive set of features, it has some limitations:

- **No Real Payment Integration:** The current system simulates payment without integrating actual payment gateways (e.g., Stripe, bKash, Nagad). The `payment_method` field stores the method name but does not process transactions.
- **Cold Start Latency:** The Render free tier puts the API server to sleep after inactivity, causing initial requests to take 2-5 seconds to respond. This would be resolved with a paid hosting plan.
- **No Push Notifications:** While notification preference toggles exist in the settings, actual push notification delivery (e.g., via Firebase Cloud Messaging) is not implemented.
- **No Real-Time Order Tracking:** Order status updates are manual (admin-driven). Real-time WebSocket-based order tracking is not yet implemented.
- **Limited Offline Support:** The app requires an active internet connection for all operations. Offline caching of previously loaded data is not implemented.
- **Dark Mode Not Fully Implemented:** While the dark mode toggle exists in settings, the actual dark theme color scheme is not fully designed in the Flutter app.
- **No Automated Testing:** The project lacks comprehensive unit tests, widget tests, and integration tests for both frontend and backend.

- **Single Language:** Despite the language preference setting, the app currently supports English only.

5.3 Future Work

The following enhancements are planned for future iterations of Aura Brew:

- **Payment Gateway Integration:** Integrate bKash, Nagad, and card payment processing using Stripe or SSLCommerz for real transactions.
- **Push Notifications:** Implement Firebase Cloud Messaging (FCM) for real-time order status updates, promotional offers, and new product alerts.
- **Real-Time Order Tracking:** Add WebSocket-based communication (Django Channels) for live order status updates between the kitchen and customer app.
- **Offline Mode:** Implement local caching using SQLite (sqflite) or Hive in Flutter to allow menu browsing and order history viewing without internet.
- **Dark Theme:** Design and implement a complete dark color scheme using `AppTheme.darkTheme` for Material 3.
- **Multi-Language Support:** Implement Flutter's intl package for full Bengali/English localization based on the stored language preference.
- **iOS Deployment:** Publish the app on the Apple App Store alongside the existing Android distribution.
- **AI-Powered Recommendations:** Use machine learning to suggest drinks based on user purchase history, time of day, and weather conditions.
- **QR Code Ordering:** Allow in-store customers to scan a table QR code to place orders directly from their phones.
- **Automated Testing Suite:** Write comprehensive unit tests (Dart test, pytest), widget tests (Flutter test), and integration tests to ensure code quality.
- **Social Authentication:** Implement Google Sign-In and Apple Sign-In as alternative authentication methods.

References

- [1] Flutter Documentation. Official Flutter framework documentation. Available at: <https://docs.flutter.dev/>
- [2] Django REST Framework Documentation. Available at: <https://www.django-rest-framework.org/>
- [3] Dart Programming Language. Official Dart language documentation. Available at: <https://dart.dev/>
- [4] Django Project. Official Django web framework documentation. Available at: <https://docs.djangoproject.com/>
- [5] PostgreSQL Documentation. Advanced open-source relational database. Available at: <https://www.postgresql.org/docs/>
- [6] Provider Package. State management solution for Flutter. Available at: <https://pub.dev/packages/provider>
- [7] Google Fonts. Open-source font library. Available at: <https://fonts.google.com/>
- [8] Render Cloud Platform. PaaS hosting service documentation. Available at: <https://render.com/docs>
- [9] Cloudinary. Cloud-based image and video management. Available at: <https://cloudinary.com/documentation>
- [10] Neon.tech. Serverless PostgreSQL platform. Available at: <https://neon.tech/docs>
- [11] Django Jazzmin. Admin UI customization package. Available at: <https://django-jazzmin.readthedocs.io/>
- [12] WhiteNoise. Simplified static file serving for Python web apps. Available at: <http://whitenoise.evans.io/>
- [13] Mozilla Developer Network (MDN). Web API and HTTP Documentation. Available at: <https://developer.mozilla.org/>